

**DIAGNÓSTICO TÉCNICO PROFESIONAL
PROYECTO CLÍNICAAPP**

Análisis técnico de un proyecto Spring Boot existente

Luis Cardenas

Cristobal Villamil

Daniel Gutierrez

Jose Castillo

9 de Agosto del 2026

Java 17 · Spring Boot 3.2.1 · MongoDB · Spring Security

DIAGNÓSTICO TÉCNICO PROFESIONAL: PROYECTO CLÍNICAAPP

1. Descripción del Proyecto

El backend auditado pertenece al sistema ClínicaApp, un ecosistema de software modular diseñado para la administración de clínicas veterinarias bajo un esquema multiempresa. La solución permite conectar diversas sucursales clínicas, gestionar propietarios de mascotas, historiales clínicos, coordinar citas médicas, procesar pagos y automatizar alertas de inasistencias o análisis predictivos de salud animal empleando algoritmos de clasificación de Machine Learning.

Componentes y Tecnologías Principales

Framework: Spring Boot 3.2.1

Versión de Java: Java 17

Persistencia de Datos: MongoDB (Motor NoSQL)

Acceso a Datos: Spring Data MongoDB (mediante MongoRepository y drivers nativos)

Arquitectura: Híbrida. Funciona como servidor MVC clásico mediante Thymeleaf para el panel interno web (usuarios administrativos, recepcionistas y veterinarios), y expone servicios REST a través de controladores dedicados para el consumo de la aplicación móvil de los clientes.

Integraciones y Librerías de Terceros:

Stripe Java SDK (v22.8.0): Gestión y simulación de procesamiento de cobros para las citas médicas.

Twilio Java SDK (v8.29.0): Envío de alertas y confirmaciones mediante SMS y WhatsApp.

Weka Stable Library (v3.8.6): Motor de minería de datos utilizado para cargar archivos de entrenamiento .arff y ejecutar modelos guardados (.model) de predicción de inasistencia a citas y riesgos articulares.

Groq API Client: Inferencia y pre-diagnósticos preliminares basados en inteligencia artificial (Llama 3).

OpenHTMLtoPDF (v1.0.10): Renderizado de plantillas HTML en documentos PDF de calidad para recetas y reportes clínicos.

Spring Security 6: Autenticación local mediante formulario, OAuth2 de terceros (Google y GitHub) y flujos complementarios para verificación biométrica/facial.

2. Análisis de Arquitectura

El backend de ClínicaApp estructura su código fuente agrupando sus clases en paquetes funcionales bajo el prefijo com.clinicaapp. Sin embargo, la auditoría detallada de sus componentes revela una baja cohesión y un fuerte acoplamiento entre la capa de presentación y la capa de acceso a datos.

mermaid

graph TD

Client[Cliente / Vista Thymeleaf / REST Client] --> Controllers[Capa Controller]

Controllers -->|Bypass directo detectado| Repositories[Capa Repository]

Controllers --> Services[Capa Service / Impl]

Services --> Repositories

Repositories --> MongoDB[(MongoDB Engine)]

2.1 Mapeo de Componentes y Acoplamiento Directo

Hecho: Los controladores encargados del flujo principal de datos omiten la capa de servicios intermediaria y consumen directamente las interfaces de la capa Repository.

Evidencia:

En

UsuarioController.java, se realiza la inyección directa de repositorios mediante @Autowired:

UsuarioRepository usuarioRepository (Línea 80)

CitaRepository citaRepository (Línea 82)

ResenaRepository resenaRepository (Línea 84)

VisitaRepository visitaRepository (Línea 86)

AnuncioGlobalRepository anuncioGlobalRepository (Línea 88)

De forma similar,

ClinicaController.java inyecta directamente 8 repositorios: ClinicaRepository, UsuarioRepository, VisitaRepository, MascotaRepository, CitaRepository, ProductoRepository, ExamenLaboratorioRepository y AnuncioGlobalRepository (Líneas 66 a 98).

Hallazgo: Existe un alto acoplamiento entre la capa de controladores y el modelo físico de almacenamiento. Si se requiere modificar la lógica de persistencia o incorporar validaciones de negocio previas a la escritura, es necesario alterar directamente los controladores de presentación web. Esto reduce drásticamente la mantenibilidad de la aplicación.

Recomendación: Refactorizar los controladores para que interactúen únicamente con interfaces del paquete service, encapsulando las invocaciones a los repositorios dentro de las implementaciones (impl) del servicio.

2.2 Cohesión y Tamaño de Clases

Hecho: Los controladores concentran lógica de transformación, orquestación y validaciones complejas de datos.

Evidencia:

UsuarioController.java posee un tamaño de 2,016 líneas de código y

ClinicaController.java tiene 1,211 líneas.

Hallazgo: Violación al principio de responsabilidad única (SRP). Los controladores no solo reciben peticiones HTTP y redireccionan vistas; también ejecutan formateo de textos, conversiones a DTOs y lógica transaccional que debería pertenecer a la capa Service.

3. Análisis de Configuración

La configuración técnica reside de manera conjunta en los archivos

application.properties y

application.yml.

3.1 Manejo de Secretos e Interpolación

Hecho: Las llaves privadas, tokens de Twilio y accesos a la base de datos se leen del sistema operativo en tiempo de ejecución utilizando placeholders como `stripe.api.key.secret=${STRIPE_SECRET_KEY:}` o `spring.data.mongodb.uri=${MONGODB_URI:mongodb://localhost:27017/clinica_veterinaria_db}`.

Fortaleza: Se implementa una correcta práctica de desacoplamiento de secretos conforme al principio de 12-Factor App, evitando almacenar credenciales reales de producción en el repositorio de Git.

3.2 Separación de Perfiles

Hecho: No existen archivos complementarios para entornos de desarrollo, pruebas o producción (tales como `application-dev.properties` o `application-test.properties`).

Hallazgo: La ausencia de perfiles dedicados impide configurar comportamientos diferentes de forma limpia (por ejemplo, utilizar bases de datos Mock para pruebas locales sin alterar los strings de conexión compartidos).

4. Manejo de Errores y Excepciones

El backend carece de una estrategia estructurada y unificada para procesar, traducir y responder ante condiciones de error imprevistas.

4.1 Análisis de la Intercepción de Errores

Hecho: El único componente anotado con `@ControllerAdvice` es

`GlobalControllerAdvice.java`. Al analizar su estructura interna, se observa que no posee anotaciones `@ExceptionHandler` para procesar excepciones. Su única función es rellenar atributos del modelo de Thymeleaf en las vistas HTML a través del método `addNotificationsToModel` anotado con `@ModelAttribute` (Línea 29).

Hallazgo: Ante fallos de negocio (como buscar un ID no existente de mascota, o enviar tipos de datos incompatibles en una petición JSON de la API REST), el servidor no intercepta el error. Dependiendo del flujo, el cliente recibe:

Una redirección con parámetros URL (ej: `redirect:/buscar-clinicas?error=notfound` en `HomeController.java:L112`).

Un error nativo de Spring Boot (Whitelabel Error Page) o stacktraces detallados en formato HTML/JSON.

Recomendación: Implementar una clase centralizada anotada con `@RestControllerAdvice` y métodos `@ExceptionHandler` que capturen excepciones específicas del sistema (como `NoSuchElementException` o `IllegalArgumentException`) y devuelvan una respuesta uniforme en formato JSON con la siguiente estructura: `{"status": 404, "error": "Recurso no encontrado", "timestamp": "..."}.`

4.2 Falta de Validaciones Automáticas con Bean Validation

Hecho: Modelos de persistencia como

`Cita.java` y estructuras DTO como

`UsuarioRegistroDTO.java` carecen de anotaciones de validación declarativa (por ejemplo, `@NotNull`, `@NotBlank`, `@Size` o `@Email` de `jakarta.validation.constraints`).

Impacto: Los controladores procesan objetos entrantes sin validar previamente sus propiedades de manera formal. Si se envían datos nulos o vacíos en peticiones REST, estas son procesadas y almacenadas directamente en la base de datos MongoDB, forzando a escribir código imperativo redundante en los controladores para evitar inconsistencias.

5. Seguridad Básica

El análisis de seguridad del archivo

SecurityConfig.java y del resto de configuraciones de red expone debilidades severas que comprometen la confidencialidad e integridad del sistema.

5.1 Deshabilitación Global de CSRF (Cross-Site Request Forgery)

Hecho: En

SecurityConfig.java:L53, se desactiva por completo la validación CSRF del framework:

```
java
```

```
.csrf(csrf -> csrf.disable())
```

Hallazgo: Dado que la aplicación utiliza autenticación mediante sesiones clásicas basadas en cookies (JSESSIONID) para los roles web (ROLE_ADMIN, ROLE_CLINICA, ROLE_VETERINARIO), deshabilitar CSRF representa un riesgo crítico de seguridad. Si un usuario tiene una sesión activa en la aplicación e ingresa a un sitio web de terceros malicioso, dicho sitio puede realizar peticiones destructivas (como el borrado o alteración de datos) hacia el backend de ClínicaApp, y el navegador adjuntará de manera automática la cookie de sesión del usuario.

Recomendación: La protección CSRF debe permanecer activa para todas las peticiones procesadas a través de sesiones web (Thymeleaf), pudiendo exceptuar únicamente los endpoints de la API REST que operan bajo esquemas de autenticación stateless.

5.2 Exposición y Control del PIN de Administración Hardcodeado

Hecho: El controlador

ApiSystemController.java:L121 valida los intentos de verificación de red de dispositivos cliente mediante una comparación directa en código de un PIN estático:

```
java
```

```
if ("1428".equals(pin)) {  
  
    deviceTracker.verifyIpPin(request.getRemoteAddr());  
  
    response.put("success", true);  
  
}
```

Hallazgo: El PIN de seguridad "1428" está escrito directamente en el código de producción. Además, al cruzar esta configuración con

SecurityConfig.java:L98, se observa que la ruta /api/ está configurada como pública/permitida (permitAll()) para habilitar llamadas de la aplicación móvil:

```
java
    .requestMatchers(HttpMethod.POST,
        "/registro",
        "/forgot-password",
        "/reset-password",
        "/auth/login-facial",
        "/api/\\\" // <-- Permitido de manera pública
    ).permitAll()
```

Esto significa que el endpoint del sistema /api/system/verify-pin es completamente accesible desde internet sin requerir autenticación previa. Cualquier atacante externo que descubra el PIN estático "1428" puede enviar una solicitud POST para verificar y autorizar el acceso de su dirección IP, evadiendo la pantalla de bloqueo y las restricciones de red del NetworkDeviceFilter.

Recomendación: Mover el PIN a las propiedades cifradas de configuración y requerir que los endpoints administrativos expuestos bajo /api/system/ estén protegidos por un token de acceso seguro o restricciones de rol de administrador.

5.3 Configuración Excesivamente Permisiva de CORS

Hecho: Se implementa una configuración CORS universal en

WebConfig.java:L22:

```
java
    registry.addMapping("\\")
        .allowedOrigins("\\*")
        .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders("\\*");
```

Adicionalmente, controladores REST individuales como

ApiCitaController.java:L15 y

ApiMascotaController.java:L15 replican explícitamente la anotación de origen universal `@CrossOrigin(origins = "*")`.

Hallazgo: Habilitar el comodín `*` permite que cualquier aplicación basada en navegador web, independientemente de su origen de host, consulte los datos privados de los clientes, mascotas e historiales de ClínicaApp.

Recomendación: Definir una lista blanca estricta con los hosts autorizados de la empresa para consumir la API.

6. Documentación de la API

Hecho: Al analizar el archivo

pom.xml, se corrobora que no existen dependencias referentes a OpenAPI o Swagger UI (tales como springdoc-openapi-starter-webmvc-ui). Tampoco existen clases de configuración de documentación ni anotaciones descriptivas sobre los endpoints REST expuestos en el subpaquete rest.

Hallazgo: La documentación de la API es inexistente en el código.

Impacto: Los equipos encargados del desarrollo cliente (móvil y web externo) deben analizar directamente el código Java de los controladores REST para determinar la estructura de los payloads esperados y los códigos de respuesta HTTP de los endpoints.

7. Pruebas Automatizadas

El proyecto no cumple con un esquema estándar de pruebas de software, lo que impide asegurar la calidad y estabilidad de la lógica de negocio durante futuras actualizaciones o refactorizaciones.

7.1 Análisis del Único Test del Proyecto

Hecho: El único test del proyecto reside en

ClinicaappApplicationTests.java:

```
java
```

```
@SpringBootTest
```

```
class ClinicaappApplicationTests {
```

```
    @Test
```

```
    void contextLoads() {}
```

```
}
```

Hallazgo: No existen pruebas unitarias para validar las complejas reglas de negocio de asignación de citas, validación de historiales ni inferencia predictiva con Weka.

7.2 Falla Crítica en Fase de Test por Acoplamiento Físico de Base de Datos

Hecho: Al ejecutar las pruebas automatizadas del proyecto en el entorno de desarrollo mediante la herramienta de empaquetado de Maven (mvnw test), la suite de pruebas falla y provoca un error de construcción general (BUILD FAILURE).

Causa Raíz: Al utilizar la anotación @SpringBootTest, Spring Boot arranca el contexto completo de la aplicación, lo que a su vez inicializa el componente de carga inicial de datos

DataSeeder.java. Este componente intenta realizar una consulta a MongoDB (repository.count()). Como no existe un servicio de MongoDB en ejecución en el puerto 27017 de la máquina de compilación local, se genera un error de conexión persistente y se agota el tiempo de espera por defecto:

```
bash
```

Caused by: com.mongodb.MongoTimeoutException: Timed out after 30000 ms while waiting for a server that matches ReadPreferenceServerSelector{readPreference=primary}

Esto confirma la ausencia total de un entorno de pruebas aislado (empleando mocking de repositorios con @MockBean o el uso de bases de datos embebidas para la fase de pruebas).

8. Matriz de Madurez

Dimensión	Inexistente	Deficiente	Aceptable	Buena práctica
Separación de capas		X		
Gestión de configuración			X	
Manejo de errores		X		
Seguridad básica		X		
Documentación de API	X			
Pruebas automatizadas		X		

Justificación Técnica:

Separación de capas: Evaluado como Deficiente debido al acoplamiento directo entre los controladores web y las interfaces de acceso a datos de MongoDB, saltándose el uso de la capa de servicios en controladores principales.

Gestión de configuración: Evaluado como Aceptable. Utiliza la interpolación correcta de variables para no exponer contraseñas y llaves de servicios de pago en texto plano en el repositorio, pero carece de separación por perfiles físicos de ejecución.

Manejo de errores: Evaluado como Deficiente por no contar con una clase unificada de control de excepciones REST, exponiendo stacktraces JVM detallados y delegando la captura de fallos de forma manual en bloques try-catch dentro de los controladores.

Seguridad básica: Evaluado como Deficiente debido a tres fallos concretos: desactivación global de la protección CSRF en flujos web basados en sesión, asignación de CORS universal en todos los endpoints, y la presencia de un PIN administrativo estático expuesto en código público sin autenticación.

Documentación de API: Evaluado como Inexistente. No contiene dependencias, anotaciones ni consolas Swagger mapeadas en el proyecto.

Pruebas automatizadas: Evaluado como Deficiente. Se limita a un único test de carga de contexto que falla al intentar compilar en un entorno sin un servidor de MongoDB activo.

9. Propuestas de Mejora Priorizadas

A continuación, se listan los problemas detectados y las respectivas recomendaciones de refactorización priorizadas:

Prioridad Alta (Seguridad y Compilación)

Remover PIN Hardcodeado y Proteger Endpoint de Sistema:

Hallazgo: PIN "1428" en

ApiSystemController.java:L121 bajo la ruta pública /api/.

Solución Propuesta: Mover la validación del PIN hacia una propiedad configurable e inyectada mediante Spring (@Value("\${system.security.verificaton-pin}")). Adicionalmente, proteger los endpoints de monitoreo de /api/system/ restringiendo su acceso en

SecurityConfig.java únicamente a peticiones autenticadas de administradores.

Reconfigurar CSRF y CORS:

Hallazgo: CORS con wildcard * y CSRF deshabilitado en toda la aplicación.

Solución Propuesta: Habilitar la protección CSRF en

SecurityConfig.java para los flujos basados en páginas web clásicas de Thymeleaf. Restringir la configuración de CORS en

WebConfig.java y en las anotaciones @CrossOrigin de los controladores REST, asignando dominios de confianza explícitos.

Establecer un Perfil y Entorno de Pruebas Aislado:

Hallazgo: Falla de Maven en el comando mvnw test por dependencia directa de un servidor local de MongoDB activo.

Solución Propuesta: Configurar un perfil exclusivo de pruebas (@ActiveProfiles("test")).

Modificar

ClinicaappApplicationTests.java para utilizar una base de datos en memoria para pruebas (como una base de datos embebida de MongoDB) o aplicar mocks de los repositorios implicados durante la fase de carga inicial (DataSeeder) utilizando la anotación @MockBean en la clase de test.

Prioridad Media (Mantenibilidad y Robustez)

Implementar @RestControllerAdvice y Validar Modelos:

Hallazgo: Falta de control de errores REST y nula validación automática de datos de entrada.

Solución Propuesta: Crear una clase de manejo global de excepciones anotada con `@RestControllerAdvice` e implementar interceptores `@ExceptionHandler` para retornar mensajes informativos en formato JSON estructurado. Incorporar dependencias de validación Bean Validation en el pom.xml y anotar campos críticos (como email y password en UsuarioRegistroDTO) para forzar validaciones automáticas mediante la anotación `@Valid` en las firmas de los métodos de los controladores.

Desacoplar los Repositorios de la Capa de Controladores:

Hallazgo: Uso directo de Repository en controladores web complejos.

Solución Propuesta: Migrar las interacciones directas con las interfaces de repositorio a sus respectivos servicios de negocio (Service), de forma que los controladores solo se encarguen de estructurar las respuestas HTTP delegando todo procesamiento y persistencia a la capa intermedia.

Prioridad Baja (Documentación y Estándares)

Integrar OpenAPI / Swagger UI:

Hallazgo: Ausencia de especificaciones de contratos de los endpoints del API Rest.

Solución Propuesta: Añadir la dependencia de Springdoc OpenAPI en el archivo pom.xml para habilitar la generación automatizada de Swagger UI, permitiendo interactuar con los endpoints desde el navegador en un ambiente de desarrollo.

10. Evidencias y Capturas Recomendadas

Para justificar de forma gráfica los hallazgos en la entrega final del diagnóstico del proyecto, se sugiere incluir capturas de pantalla de los siguientes puntos:

Estructura del Paquete del Backend: Árbol de directorios mostrando la organización de paquetes del código fuente de Spring Boot para ilustrar la ubicación de las capas.

Bypass del Service en Controlador: Vista del archivo

UsuarioController.java a nivel de la declaración de variables autovinculadas para evidenciar la inyección de repositorios.

PIN de Red en Texto Claro: Vista del archivo

ApiSystemController.java en la estructura condicional de verificación de PIN ("1428".equals(pin)).

Configuración de CORS Abierto: Vista en el editor de la clase

WebConfig.java resaltando la llamada .allowedOrigins("*").

Desactivación Global de CSRF: Imagen de la llamada .csrf(csrf -> csrf.disable()) en el método filterChain de

SecurityConfig.java.

Log de Consola del Fallo del Test: Captura del output del terminal tras ejecutar el comando mvnw test, mostrando específicamente la traza de error de com.mongodb.MongoTimeoutException que impide compilar el proyecto exitosamente.

Acceso al proyecto y presentación web

Como complemento al presente diagnóstico técnico, se dispone de una página web donde se presenta de manera visual e interactiva la información analizada, incluyendo la arquitectura, hallazgos, aspectos de seguridad, configuración, pruebas, evidencias y recomendaciones de mejora.

Presentación web del diagnóstico:

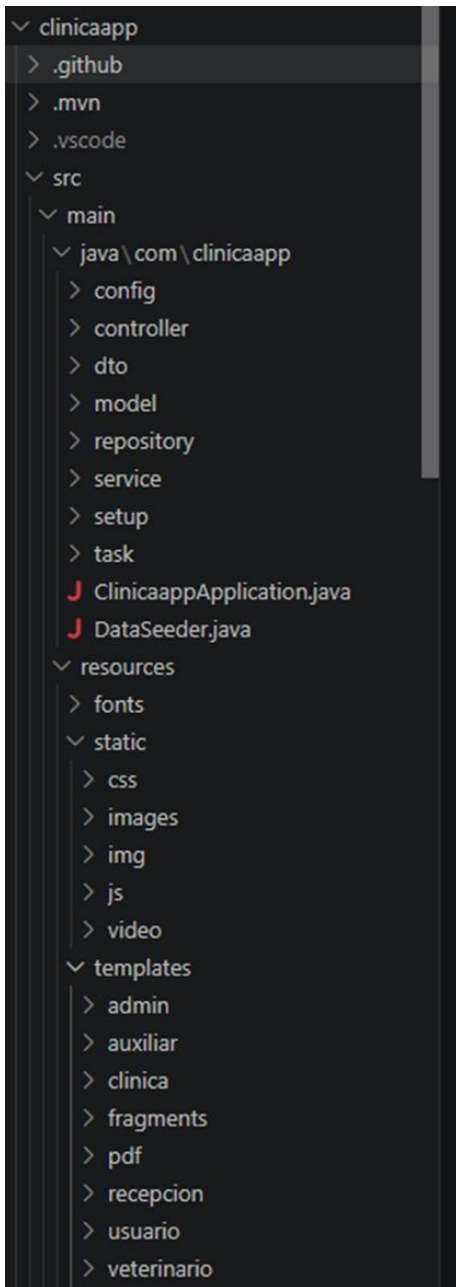
<https://cardenaswalker2.github.io/Tareaweb1/>



Asimismo, se encuentra disponible el repositorio del proyecto ClínicaApp, donde puede consultarse el código fuente y la estructura del sistema analizado:

Repositorio del proyecto ClínicaApp:

<https://github.com/cardenaswalker2/ClinicaApp>



```
@Controller
@RequestMapping("/usuario")
public class UsuarioController {

    @Autowired
    private ComunidadPetService comunidadPetService;

    @Autowired
    private MensajeChatRepository chatRepository;

    @Autowired
    private IUsuarioService usuarioService;
    @Autowired
    private UsuarioRepository usuarioRepository;
    @Autowired
    private CitaRepository citaRepository;
    @Autowired
    private ResenaRepository resenaRepository;
    @Autowired
    private VisitaRepository visitaRepository;
    @Autowired
    private com.clínicaapp.repository.AnuncioGlobalRepository anuncioGlobalRepository;

    @Autowired
    private IMascotaService mascotaService;
    @Autowired
    private ICitaService citaService;
    @Autowired
    private IClinicaService clinicaService;
    @Autowired
    private IVisitaService visitaService;
    @Autowired
    private IPdfService pdfService;
    @Autowired
    private IServicioService servicioService;
    @Autowired
    private IEmailService emailService;
    @Autowired
    private IResenaService resenaService;
    @Autowired
    private IPredictiveHealthService predictiveHealthService;
    @Autowired
    private IJuegoService juegoService;
```

```
@PostMapping("/verify-pin")
public Map<String, Object> verifyPin(@RequestParam("pin") String pin, jakarta.servlet.http.HttpServletRequest request) {
    Map<String, Object> response = new HashMap<>();
    if ("1428".equals(pin)) {
        deviceTracker.verifyIpPin(request.getRemoteAddr());
        response.put(key: "success", value: true);
    } else {
        response.put(key: "success", value: false);
    }
    return response;
}
```

```

@Override
public void addCorsMappings(org.springframework.web.servlet.config.annotation.CorsRegistry registry) {
    registry.addMapping(pathPattern: "/*")
        .allowedOrigins(...origins: "*")
        .allowedMethods(...methods: "GET", "POST", "PUT", "DELETE", "OPTIONS")
        .allowedHeaders(...headers: "*");
}
}

```

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .cors(cors -> {}))
        .csrf(csrf -> csrf.disable())
        .addFilterBefore(networkDeviceFilter, beforeFilter: org.springframework.security.web.authentication.
UsernamePasswordAuthenticationFilter.class)
        .addFilterBefore(maintenanceFilter, beforeFilter: org.springframework.security.web.authentication.
UsernamePasswordAuthenticationFilter.class)
}

```

```

[INFO] Results:
[INFO]
[ERROR] Errors:
[ERROR] ClinicaappApplicationTests.contextLoads » IllegalStateException Failed to load ApplicationContext for [WebMergedContextConfiguration@
7ab1b10 testClass = com.clinicaapp.ClinicaappApplicationTests, locations = [], classes = [com.clinicaapp.ClinicaappApplication], conte
tInitializerClasses = [], activeProfiles = [], propertySourceDescriptors = [], propertySourceProperties = ["org.springframework.boot.t
st.context.SpringBootTestContextBootstrapper=true"], contextCustomizers = [org.springframework.boot.test.context.filter.ExcludeFilterC
ntextCustomizer@33bc72d1, org.springframework.boot.test.json.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCus
omizer@353352b6, org.springframework.boot.test.mock.mockito.MockitoContextCustomizer@0, org.springframework.boot.test.web.client.TestR
stTemplateContextCustomizer@3e08ff24, org.springframework.boot.test.autoconfigure.actuate.observability.ObservabilityContextCustomizerF
actory$DisableObservabilityContextCustomizer@1f, org.springframework.boot.test.autoconfigure.properties.PropertyMappingContextCustomiz
r@0, org.springframework.boot.test.autoconfigure.web.servlet.WebDriverContextCustomizer@312aa7c, org.springframework.boot.test.context
SpringBootTestAnnotation@3e2c8a36], resourceBasePath = "src/main/webapp", contextLoader = org.springframework.boot.test.context.Spring
ootContextLoader, parent = null]
[INFO]
[ERROR] Tests run: 1, Failures: 0, Errors: 1, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 41.242 s
[INFO] Finished at: 2026-08-09T21:40:15-05:00
[INFO] -----
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire-plugin:3.1.2:test (default-test) on project clinicaapp:
[ERROR] Please refer to C:\Users\USUARIO\Downloads\clinicaapp\clinicaapp\target\surefire-reports for the individual test results.
[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
C:\Users\USUARIO\Downloads\clinicaapp\clinicaapp>

```

